CSE-3527 Compiler

Israt Binteh Habib
Lecturer
Department of CSE, IIUC

Compiler

- *A compiler is a program that can read a program in one language-the source language -and translate it into an equivalent program in another language- the target language.*

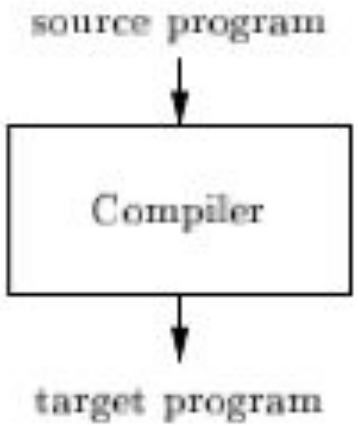

Figure 1.1: A compiler

Interpreter

- *An interpreter usually give better error diagnostics than a compiler, because it executes the source program statement by statement.*

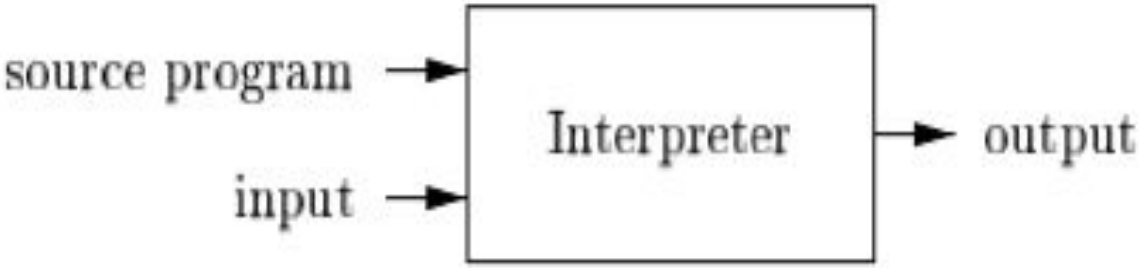

Figure 1.3: An interpreter

Java language processors

- Combination of compilation and interpretation.
- A Java source program first compiled into an intermediate form called bytecodes. The bytecodes are then interpreted by a virtual machine.
- Bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.

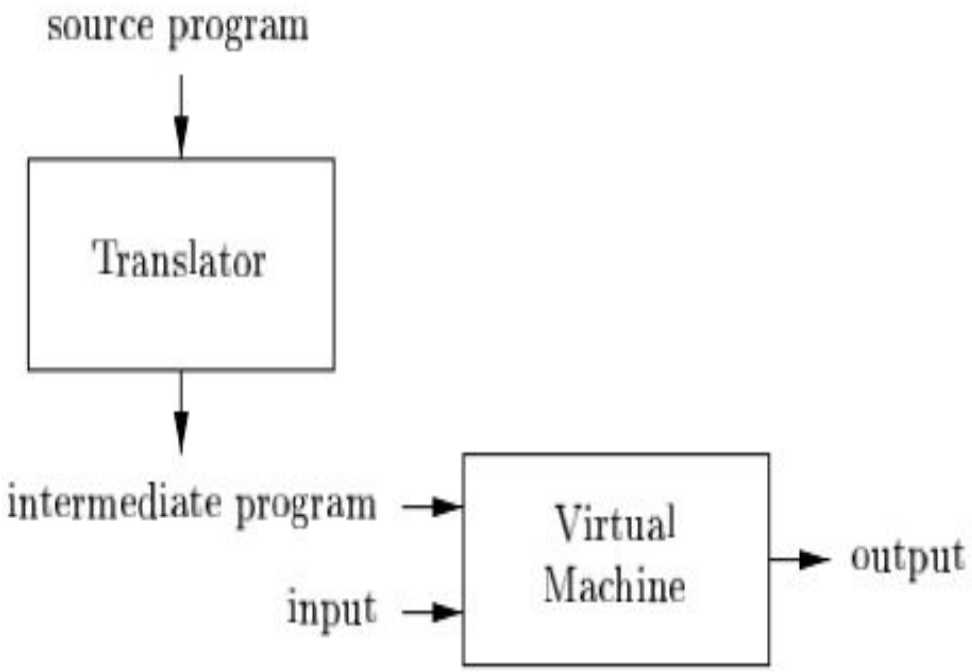

Figure 1.4: A hybrid compiler

Language Processing System

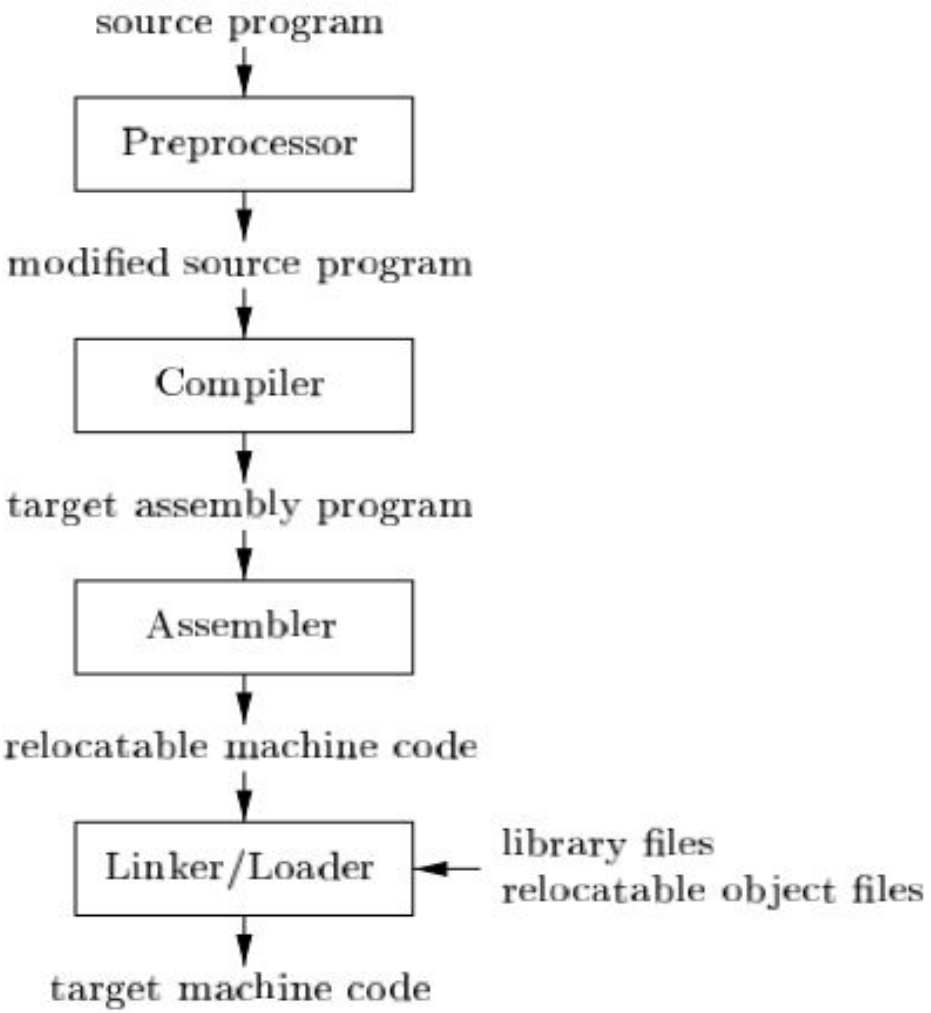

Figure 1.5: A language-processing system

Structure of a Compiler

- *There are two parts to this mapping: analysis and synthesis.*
- *The analysis part breaks up the source program into constituent pieces and imposes a grammatical structure on them. It then uses this structure to create an intermediate representation of the source program.*
- *The synthesis part constructs the desired target program from the intermediate representation and the information in the symbol table.*

Structure of a Compiler

- Lexical Analysis: The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called lexemes. For each lexeme, the lexical analyzer produces as output a token of the form: <token-name, attribute-value>
- Syntax Analysis: The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.
- Semantic Analysis: The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.

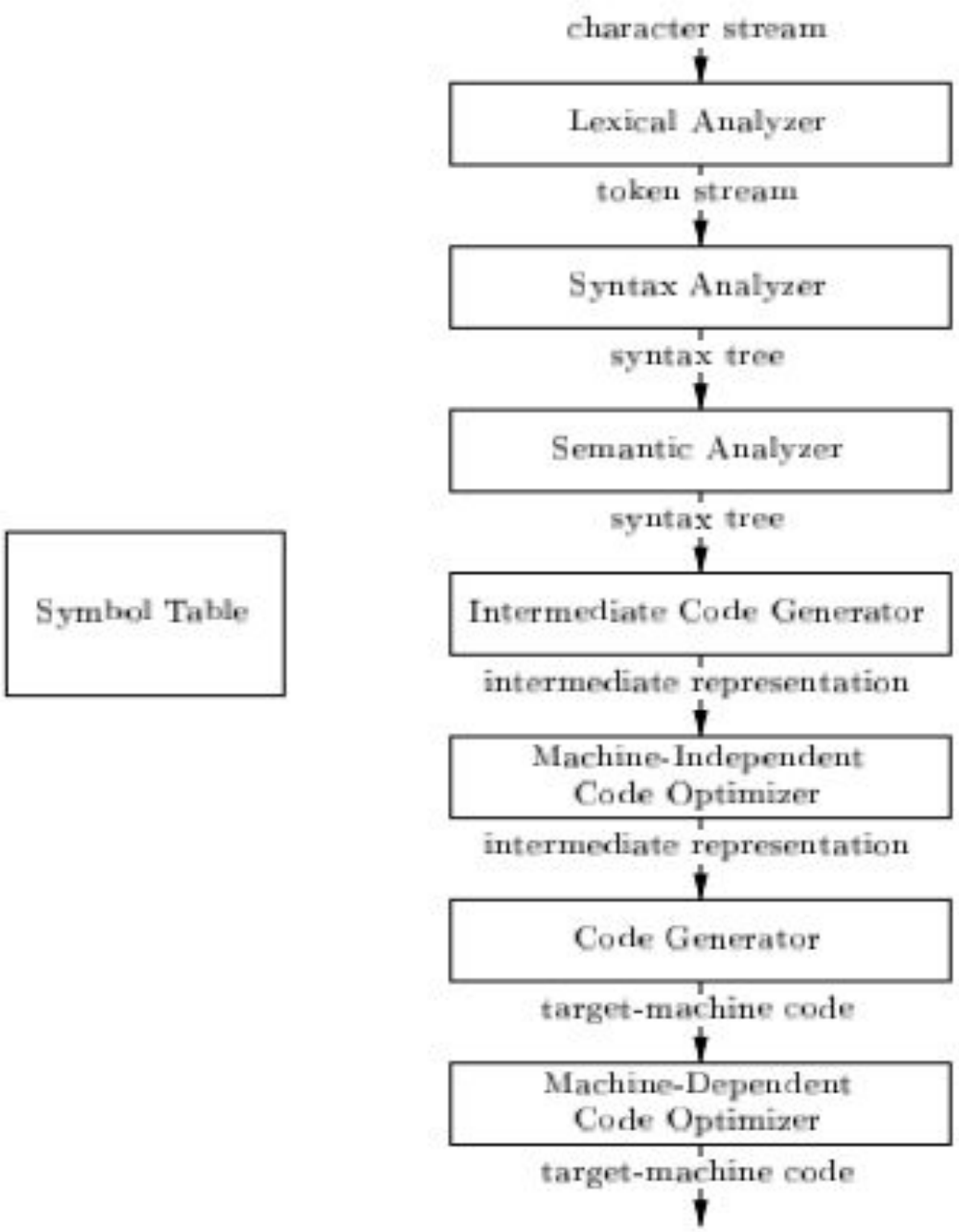

Figure 1.6: Phases of a compiler

Structure of a Compiler

- Intermediate Code Generation: a compiler may construct one or more intermediate representations, which should be easy to produce and it should be easy to translate into the target machine.
- Code Optimization: The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result.
- Code Generation: The code generator takes as input an intermediate representation of the source program and maps it into the target language.

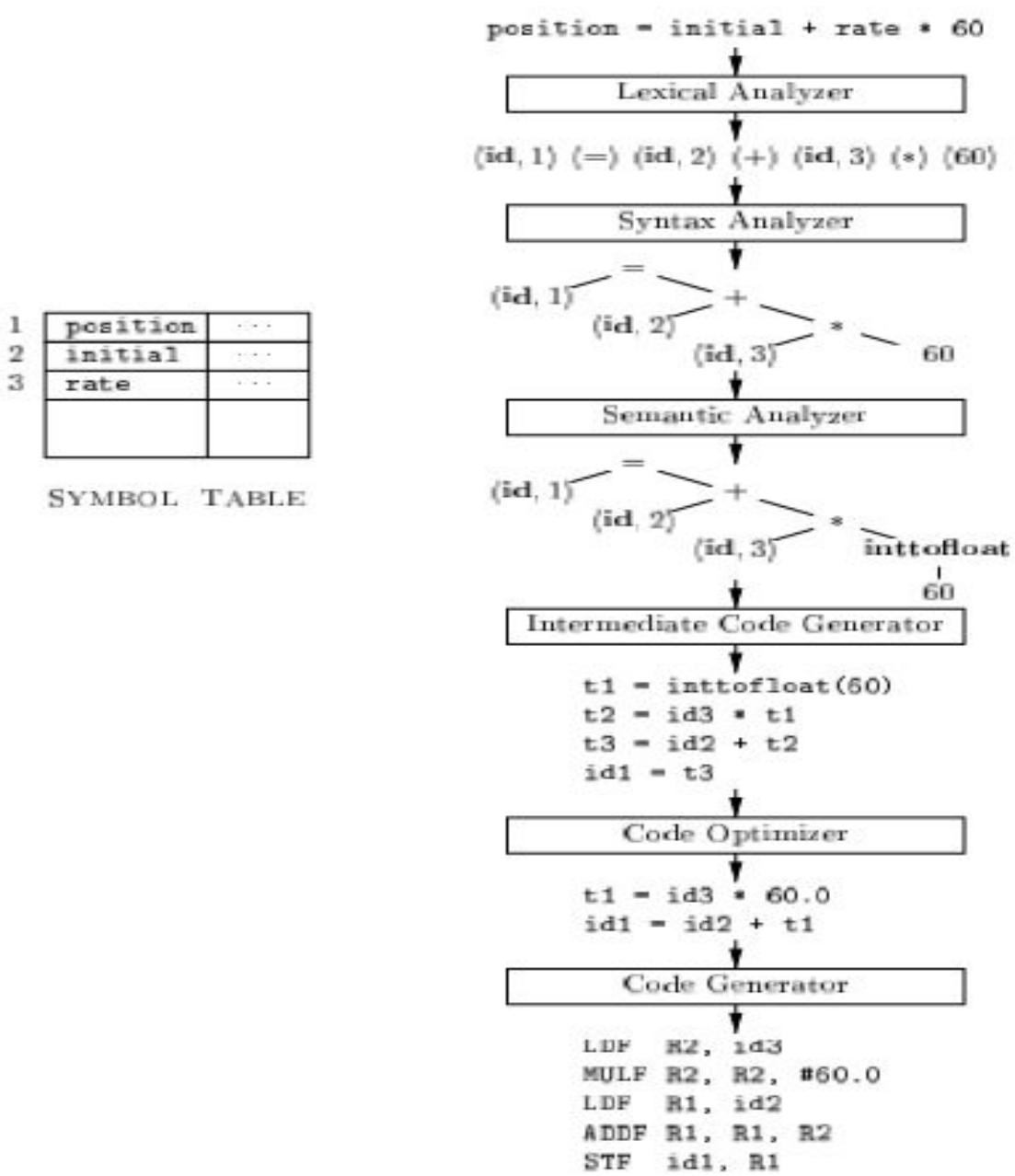

Figure 1.7: Translation of an assignment statement

Application of Compiler

- Implementation of High-Level Programming
- Optimizations for Computer Architectures
- Design of New Computer Architectures
- Program Translations
- Software Productivity Tools

Homework

- Illustrate briefly different phases of the compiler for the following statement:
  i. a = a*b + c/d
  ii. p = i + c * r * 5.0
  iii. net_salary = gross_salary - basic_salary * tax_rate

- What is the difference between a compiler and an interpreter?

References

Compilers Principles, Techniques, & Tools by Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman (2nd Edition)